

Отправка форм

- Введение
- GET-запросы
- POST-запросы
- Смешанные запросы
- GET или POST
- Передача массивов через форму

Введение

Одним из основных способов передачи данных веб-сайту являются **формы**. Формы представляют специальные элементы разметки HTML, которые содержат в себе различные элементы ввода - текстовые поля, кнопки и т.д. С помощью форм мы можем ввести некоторые данные и отправить их на сервер. Задача сервера - **обработать полученные данные**.

Важно. Каждый элемент формы автоматически **становится доступен** в коде PHP сценария.

Стандартный алгоритм создания форм состоит из следующих шагов:

- создание **контейнера** `<form></form>` в разметке HTML;
- установка **метода передачи данных**. Чаще всего используются методы GET или POST;
- **установка адреса**, на который будут отправляться введенные пользователем данные формы;
- добавление в контейнер `<form>` одного или нескольких **полей ввода**.

Существует два основных механизма передачи данных из HTML-формы на сервер: **GET** и **POST**.

GET-запросы

Формы, отправляемые методом GET, передают данные через URL-адрес. Форма создает длинную строку запроса, которая отображается в логах сервера и в адресной строке браузера.

Например:

```
http://php-course/test/index.html?page=autor&name=cortazar
```

Часть строки до символа (?) — это **полный путь к файлу**, остальная часть — **передаваемые данные**. Данные разделяются на блоки "имя=значение" посредством символа &.

В примере выше мы сформировали две глобальных переменных:

- `$_GET['page'] = "autor"`
- `$_GET['name'] = "cortazar"`

Поскольку переменные отображаются в URL-адресе, страницу можно **добавить в закладки**. В некоторых случаях это может быть полезно.

Метод GET имеет свои **ограничения и особенности**:

- Ограничение на объем отправляемой информации зависит от **лимита сервера и типа браузера**. Не существует конкретной максимальной величины GET-запроса. Один сервер может принимать максимум 2 Кб, другой — 16 Кб. Средний размер запроса колеблется в пределах 4 Кб.
- Не может отправлять на сервер **двоичные данные**, например изображения или текстовые документы.
- Так как данные строки запроса видны пользователям, метод не может использоваться для отправки **конфиденциальных данных**.

example_1/form.html. GET-форма

```
<!-- атрибут name для элемента формы обязателен -->

<form action="server.php" method="get">

    Логин: <input type="text" name="login"><p>

    E-mail: <input type="email" name="email"><p>

    Пароль: <input type="password" name="pwd"><p>

    <input type="submit">

</form>
```

Алгоритм создания разметки **GET-формы**:

- в разметке HTML создали элемент **<form></form>**;
- установили метод передачи данных - **get** (все данные передаются в адресной строке);
- установили адрес, на который будут отправляться введенные данные - **example_1_2.php**;
- добавили в элемент **<form>** три поля ввода - **login, email, pwd**.

Примечание. Сейчас не разбираем вопросы вёрстки и оформления страниц.

Откройте в браузере форму директории **example_1**, заполните текстовые поля и отправьте данные формы на сервер.

Данные, отправленные пользователем, автоматически помещаются в **суперглобальный ассоциативный массив \$_GET** только при **наличии** в соответствующих элементах формы свойства **name**.

Важно. Элементы формы будут добавлены в глобальный массив, только при наличии в этих элементах свойства **name**.

Сценарий-обработчик данных GET-формы представлен в коде следующего демонстрационного примера.

example_1/server.php. Обработчик GET-формы

```
<?php

if(isset($_GET['login']) && isset($_GET['email']) &&
isset($_GET['pwd'])) {

    // данные хранятся в массиве GET

    // можно посмотреть что получили

    echo "<pre>";

    print_r($_GET); // проверяем массив $_GET

    echo "</pre>";

    // выводим полученные от пользователя данные

    echo <<<HERE

        <h2>Полученные от пользователя данные:</h2>

        Логин: {$_GET['login']} <br />

        E-mail: {$_GET['email']} <br />

        Пароль: {$_GET['pwd']}

    HERE;

}

?>
```

POST-запросы

Для того чтобы обратиться к серверу методом GET, достаточно набрать URL-адрес в адресной строке браузера. Метод POST работает по другому принципу, **данные форм передаются через HTTP-заголовки.**

Таким образом, данные, переданные методом POST, намного труднее перехватить злоумышленнику, т.к. они скрыты от непосредственного

просмотра. Передача данных методом POST считается более безопасным способом обмена информацией.

К преимуществам метода POST стоит отнести возможность передавать на сервер не только текст, но и **мультимедиа данные** (файлы изображений, аудио, видео).

Метод POST, в отличие от GET, **не устанавливает ограничения на объем передаваемой информации.**

Чтобы на сервере получить данные, которые были переданы методом, используется глобальный массив **`$_POST`**.

example_2/form.html. POST-форма

```
<!-- атрибут name для элемента формы обязателен -->
<form action="server.php" method="post">

    Логин: <input type="text" name="login"><p>

    Е-mail: <input type="email" name="email"><p>

    Пароль: <input type="password" name="pwd"><p>

    <input type="submit">

</form>
```

Алгоритм создания разметки **POST-формы**:

- в разметке HTML создали элемент **`<form></form>`**;
- установили метод передачи данных – **post**;
- установили адрес, на который будут отправляться введенные данные - **example_2_2.php**;
- добавили в элемент **`<form>`** три поля ввода - **login, email, pwd**.

Откройте в браузере форму директории **example_2**, заполните текстовые поля и отправьте данные формы на сервер.

Данные формы, отправленные пользователем, автоматически помещаются в **суперглобальный ассоциативный массив `$_POST`**

только при наличии в соответствующих элементах формы свойства **name**.

Важно. Элементы формы будут добавлены в глобальный массив, только при наличии в этих элементах свойства **name**.

Рассмотрим сценарий обработчика POST-формы.

example_2/server.php. Обработчик POST-формы

```
<?php

    if(isset($_POST['login']) && isset($_POST['email']) &&
        isset($_POST['pwd'])) {

        // данные хранятся в массиве POST

        // можно посмотреть что получили

        echo "<pre>";

        print_r($_POST); // проверяем массив $_POST

        echo "</pre>";

        // выводим полученные от пользователя данные

        echo <<<HERE

            <h2>Полученные от пользователя данные:</h2>

            Логин: {$_POST['login']} <br />

            E-mail: {$_POST['email']} <br />

            Пароль: {$_POST['pwd']}

        HERE;

    }

?>
```

GET или POST

И метод GET и метод POST создают массив, который содержит пару **ключ=значение**:

```
array( key => value, key2 => value2, key3 => value3, ...)
```

где:

- **key** - **имена элементов управления формы** (атрибут name),
- **value** - **входные данные** от пользователя.

Массивы \$_GET и \$_POST - **суперглобальные переменные**, это означает, что они всегда доступны, к ним можно получить доступ из любой функции, класса или файла, без необходимости делать что-то особенное.

Важно.

- **\$_GET массив** переменных, передается скрипту в **параметрах URL** (пользователь легко видит данные).
- **\$_POST массив** переменных, передается скрипту в **теле POST запроса** (незаметно для пользователя).

Когда использовать GET?

Информация, отправляемая из формы методом GET видна пользователю (все имена и значения переменных отображаются в URL). GET также **имеет ограничения по объему отправляемой информации**.

Средний размер запроса колеблется в пределах 2-16 Кб.

Однако, поскольку переменные отображаются в URL-адресе, страницу можно добавить в закладки. Что может пригодиться в некоторых случаях.

Важно. GET никогда не следует использовать для отправки паролей или другой конфиденциальной информации!

Когда использовать POST?

Информация, отправленная из формы методом POST не видна для других (все имена/значения встроены в тело HTTP запроса) и **не имеет ограничений о количестве информации** для отправки.

Однако, поскольку переменные не отображаются в строке URL адреса, страницу невозможно добавить в закладки.

Примечание. Разработчики предпочитают метод POST для отправки данных формы.

Смешанные запросы

При необходимости разработчик может **совместить методы** для передачи данных формы. Никто не мешает свойство **method** формы установить в **post** и при этом к свойству **action** дописать **строку запроса**.

example_3/form.html. Смешанная форма (MIX-форма)

```
<!--  
    в строку запроса можем добавить данных сколько угодно  
    и составить массив какой угодно сложности  
-->  
  
<!-- атрибут name для элемента формы обязателен -->  
  
<form
```



```

action="server.php?data[role]=moderator&data[action]=work&mood=s
uper" method="post">

    Логин: <input type="text" name="login"><p>

    E-mail: <input type="email" name="email"><p>

    Пароль: <input type="password" name="pwd"><p>

    <input type="submit">

</form>

```

Ниже приведен пример обработчика формы, выполняющей отправку смешанного запроса.

Протестируйте отправку и обработку смешанных данных файлов директории **example_3**.

example_3/server.php. Обработчик MIX-формы

```

<?php

    // проверяем массив $_GET

    echo "<pre><b>Данные массива GET</b><br />";

    print_r($_GET);

    echo "</pre>";

    // проверяем массив $_POST

    echo "<pre><b>Данные массива POST</b><br />";

    print_r($_POST);

    echo "</pre>";

    if(isset($_GET['data']['role']) &&
    isset($_GET['data']['action']) && isset($_GET['mood'])) {

        // выводим полученные от пользователя данные (массив
        GET)

        echo <<<HERE

                <h2>Полученные от пользователя данные массива

```

```
GET:</h2>

Роль: {$_GET['data']['role']} <br />

Действие: {$_GET['data']['action']} <br />

Настроение: {$_GET['mood']}

    HERE;

}

if(isset($_POST['login']) && isset($_POST['email']) &&
isset($_POST['pwd'])) {

    // выводим полученные от пользователя данные (массив
    POST)

    echo <<<HERE

        <h2>Полученные от пользователя данные массива
        POST:</h2>

        Логин: {$_POST['login']} <br />

        E-mail: {$_POST['email']} <br />

        Пароль: {$_POST['pwd']}

    HERE;

}

?>
```

Передача массивов через форму

Раньше мы рассмотрели возможность передачи массива с использованием адресной строки.

Разберем возможность работы с формами, при которой можно объединить несколько элементов формы и получить их на сервере в виде массива. Это делается с помощью манипуляций с атрибутом **name**.

Примечание. Массивы используются, например, когда в форме расположены несколько элементов типа **checkbox**, отвечающих за одно и то же поле.

Обработка элементов checkbox

Рассмотрим способ организации элементов формы, в которой пользователю требуется выбрать те языки программирования, которые он хотел бы изучать.

example_4/form.html. Создание массива из элементов checkbox

```
<form action="server.php" method="post">

    Имя: <input type="text" name="name"><p>

    Е-mail: <input type="email" name="email"><p>

    Язык программирования:<br />

    <input type="checkbox" name="lang[]" value="php" /> php <br
    />

    <input type="checkbox" name="lang[]" value="python" />
    python <br />

    <input type="checkbox" name="lang[]" value="perl" /> perl
    <br />

    <input type="checkbox" name="lang[]" value="java" /> java
    <br />

    Образование:<br />

    <input type="radio" name="edu" value="очное" /> очное <br />

    <input type="radio" name="edu" value="заочное" /> заочное
    <br />

    <input type="radio" name="edu" value="дистанционное" />
    дистанционное <br />
```

```
<input type="submit">

</form>
```

После отправки формы в сценарии-обработчике будет доступен массив **`$_POST["lang"]`** со значениями, отмеченными пользователем. Такой приём использования массивов в HTML-форме встречается довольно часто.

example_4/server.php. Получение массива элементов checkbox

```
<?php

    // вывод полученных данных массива POST

    echo "<h2>Вывод полученных данных массива POST</h2>";

    echo "<pre>";

    print_r($_POST);

    echo "</pre>";

    // вывод данных раздела Языки программирования

    echo "<h2>Вывод данных раздела Языки программирования</h2>";

    echo "<pre>";

    print_r($_POST['lang']);

    echo "</pre>";

    // вывод данных раздела Образование

    echo "<h2>Вывод данных раздела Образование</h2>";

    echo "<pre>";

    print_r($_POST['edu']);

?>
```

Обратите внимание, если вы не выбрали ни одно значение из предложенных checkbox-ов, этот параметр вовсе **не будет отправлен на сервер**. Это следует учитывать и делать дополнительную проверку на **существование переменной** в глобальном массиве.

Выполним проверку на наличие массива в принимающем скрипте:

example_5/form.html. Формирование массива элементами checkbox

```
<form action="server.php" method="post">

    <h2>Выбери хобби на букву С:</h2>

    <input type="checkbox" name="hobby[]" value="Серфинг"
    />Серфинг<br />

    <input type="checkbox" name="hobby[]" value="Сноуборд"
    />Сноуборд<br />

    <input type="checkbox" name="hobby[]" value="Скейтборд"
    />Скейтборд<br />

    <input type="checkbox" name="hobby[]" value="Страйкбол"
    />Страйкбол<br />

    <input type="checkbox" name="hobby[]" value="Сайтостроение"
    />Сайтостроение<p>

    <input type="submit">

</form>
```

example_5/server.php. Проверяем наличие массива

```
<?php

// проверка массива POST['hobby']

if (isset($_POST["hobby"])) {

    echo "<h2>Вы выбрали следующий список хобби</h2>";

    echo "<pre>";

    print_r($_POST["hobby"]);

    echo "</pre>";

} else {

    echo "<h2>Скучный вы человек...</h2>";

    echo "<a href='example_9.html'>Попробуйте еще раз
```

```
        ; ) </a>";  
  
    }  
  
?>
```

Обработка элементов select

Работа с выпадающим списком select мало чем отличается от обычных полей, тут больше отличий со стороны разметки HTML.

Для демонстрации формирования массива создадим форму **множественного выбора** (атрибут **multiple**). Атрибут множественного выбора очень важен, без него на сервер будет отправляться не массив, а простое строковое значение, но это не то, что нам нужно.

example_6/form.html. Создание массива из элемента select

```
<form action="server.php" method="post">  
  
    Логин: <input type="text" name="login"><p>  
  
    Сообщение: <input type="text" name="msg"><p>  
  
    Выберите желаемые блага <br />  
  
    (возможен множественный выбор) <p>  
  
    <select name="benefits[]" id="edu" size=5 multiple>  
  
        <option value="range-rover">Авто Range Rover Velar  
        I</option>  
  
        <option value="royal-cruiser">Яхта Royal  
        Cruiser</option>  
  
        <option value="aviator-residence">Коттедж Aviator  
        Residence</option>  
  
        <option value="roza-rossa">Дом Roza Rossa</option>  
  
        <option value="happiness">Здоровье, ум,  
        красота</option>
```

```
<option value="no-thanks">Спасибо, у меня все  
есть...</option>  
  
</select><p>  
  
<input type="submit">  
  
</form>
```

Обратите внимание, у поля **select** атрибут **name** указывается непосредственно в теге **select**, а вот **value** — значение поля, указывается внутри каждого тега **option**.

Более того, по умолчанию при выводе формы в браузере, в поле **select** всегда выбрано первое доступное значение. Чтобы этого избежать, обычно добавляют дополнительный элемент **option** в начало с пустым атрибутом **value**. Тогда пользователю нужно будет непосредственно выбрать доступную в выпадающем списке опцию.

example_6/server.php. Получение массива элементов option

```
<?php  
  
    //вывод полученных данных массива POST  
  
    echo "<h2>Массив POST содержит следующие данные</h2>";  
  
    echo "<pre>";  
  
    print_r($_POST);  
  
    echo "</pre>";  
  
    echo "<h2>Вы взалкали следующих благ...</h2>";  
  
    echo "<pre>";  
  
    print_r($_POST['benefits']);  
  
    echo "</pre>";  
  
?>
```

Обработка элементов radio

Атрибут `type` тега **input** со значением **radio** обычно используется для создания группы радиокнопок (переключателей), описывающих набор взаимосвязанных параметров. Одновременно пользователь может выбрать лишь **одну радиокнопку** из группы предложенных. Если возможен выбор одного варианта, почему радиокнопки находятся в разделе формирования массивов формы?

Ответ прост, веб разработчик должен уметь формировать сложные формы, собирать и предавать на сервер данные разных структур.

Ну, а пример, может быть такой: вам необходимо сформировать **набор критериев** для поиска информации в базе данных.

- для удобства **пользователя**, элементы формирования критерия разносите по разным группам;
- для удобства **обработки**, собираете все критерии в один массив.

Смотрим.

example_7/form.html. Создание массива из элементов `type=radio`

```
<form action="server.php" method="post">

    Представьтесь: <input type="text" name="name"><p>

    Бренд:

    Acer    <input type="radio" name="search[brand]"
value="Acer">

    Dell    <input type="radio" name="search[brand]"
value="Dell"><p>

    Операционная система (OS) :

    Windows <input type="radio" name="search[os]"
value="Windows">

    Linux   <input type="radio" name="search[os]"
value="Linux"><p>

    Объем твердотельных накопителей (SSD) :
```



```
512 <input type="radio" name="search[ssd]" value="512">
1024 <input type="radio" name="search[ssd]" value="1024"><p>
<input type="submit">
</form>
```

Теперь в обработчике формы нам доступен массив **`$_POST["search"]`** который можно использовать для формирования **запроса** к базе данных, **хранения** в логах, **передачи** в виде строки JSON или для выполнения любой другой задачи.

example_7/server.php. Получение массива элементов type=radio

```
<?php
    echo "<h2>Уважаемый, {$_POST['name']} ;) </h2>";
    echo "<h3>Мы сформировали список ваших предпочтений и готовы
начать поиск!</h3>";
    // критерий поиска
    echo "<pre>";
    print_r($_POST["search"]);
    echo "</pre>";
?>
```

Примечание. В одной из самостоятельных работ к этому уроку рассмотрим этот пример чуть более подробно.

Формирование индексных массивов из текстовых полей

Рассмотрим ещё один вариант использования возможности создания массивов в HTML-форме.

Как мы формировали массивы из чекбоксов или списков, так же можно формировать массив из, например, текстовых полей.

К сведению. В некоторых случаях при обработке принятых данных использовать **упорядоченную структуру массива** удобнее, чем набор разрозненных данных.

example_8/form_1.html. Создание индексного массива текстовых полей

```
<h1>Формирование индексного массива данных в форме</h1>

<form action="server.php" method="post">

    Логин:<input type="text" name="data[]"><p>

    E-mail:<input type="text" name="data[]"><p>

    Пароль:<input type="text" name="data[]"><p>

    <input type="submit">

</form>
```

Формирование ассоциативных массивов из текстовых полей

Из элементов текстовых полей формы формировать можно не только индексный, но и **ассоциативный** массив.

example_8/form_2.html. Создание ассоциативного массива текстовых полей

```
<h1>Формирование ассоциативного массива данных в форме</h1>

<form action="server.php" method="post">

    Логин: <input type="text" name="data[login]"><p>

    E-mail: <input type="text" name="data[email]"><p>

    Пароль: <input type="text" name="data[pwd]"><p>
```

```
<input type="submit">

</form>
```

Далее можно использовать данные сформированного **индексного** или **ассоциативного массива** на своё усмотрение.

example_8/server.php. Получение массива текстовых полей формы

```
<?php

    // выводим данные массива $_POST

    echo "Скрипт получил следующие данные";

    echo "<pre>";

    print_r($_POST);

    echo "</pre>";

?>
```

Примечание. Я намеренно не привожу примеры обработки принятых данных.

Задачей сервера является - **обработка** полученных от пользователя данных. А что дальше?

Важно. Следующий шаг — это **запись** принятых и обработанных данных либо в **файл**, либо в **базу данных** (как правило). Но это тема отдельных пар.